Vector Databases, Embeddings and RAG



SoftUni Team

Technical Trainers

Software University

SoftUni



Software University

https://about.softuni.bg

Table of Contents

1. Types of memories
2. Embeddings: Representing Meaning as Vectors
3. Vector Databases: Semantic Search at Scale
4. RAG in Action: Putting It All Together





Types of memory

How do machines remember?

Contextual (Short-Term) Memory

Software University

Definition: Keeps conversations **coherent** within a single session

Example:

```
User: "Create a Python function to calculate tax"
AI: "Here's a tax calculation function..."

User: "Now add error handling to it"
AI: "I'll add error handling to the tax function..." ✅
```

Long-Term (Persistent) Memory

Software University

Definition: Remembers **facts** and **preferences** across multiple sessions

Example:

```
Session 1:
User: "I prefer TypeScript over JavaScript"
AI: "Noted! I'll use TypeScript in future examples."

Session 2 (next day):
User: "Help me build a web API"
AI: "I'll create a TypeScript-based API for you..." ✅
```

Semantic Memory

Definition: The model's built-in **factual** and **conceptual knowledge** from training

Example:

```
User: "Explain how TCP/IP works"
AI: "TCP/IP is a protocol suite where TCP handles..." ✅

User: "What happened in tech news yesterday?"
AI: "I don't have access to recent news..." ❌
```

Working Memory

Definition: Handles multi-step **reasoning** and **problem-solving** within a single query

Example:

```
User: "Calculate the total cost for 3 servers, each with 4 CPUs at $50/month
per CPU, plus $200 storage each, for 6 months"

AI thinks step by step:
1. 3 servers × 4 CPUs = 12 CPUs total
2. 12 CPUs × $50 = $600/month for CPUs
3. 3 servers × $200 = $600/month for storage
4. Total monthly: $600 + $600 = $1,200
5. 6 months × $1,200 = $7,200 total ✅
```

External / Tool-Augmented Memory (RAG)

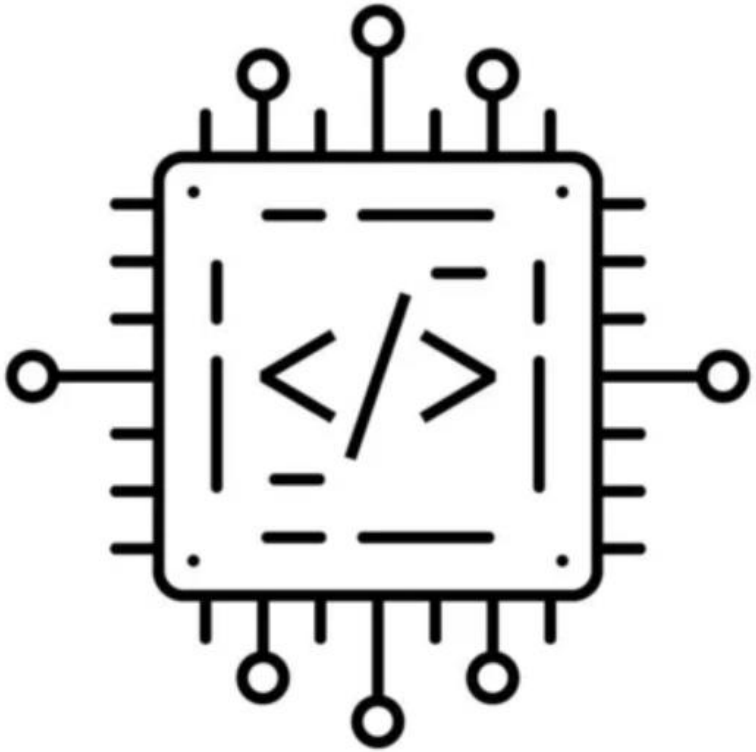
Definition: Extends AI capacity by **retrieving information** from **external sources**

Example:

```
User: "What's our return policy for electronics?"

AI Process:
1. 🔍 Search company knowledge base
2. 📄 Find return policy document
3. 🧠 Combine found info with user question
4. 💬 Generate accurate, company-specific answer

AI: "According to our policy, electronics can be returned within 30 days with
original packaging..." ✅
```



Embeddings

Representing Meaning as Vectors

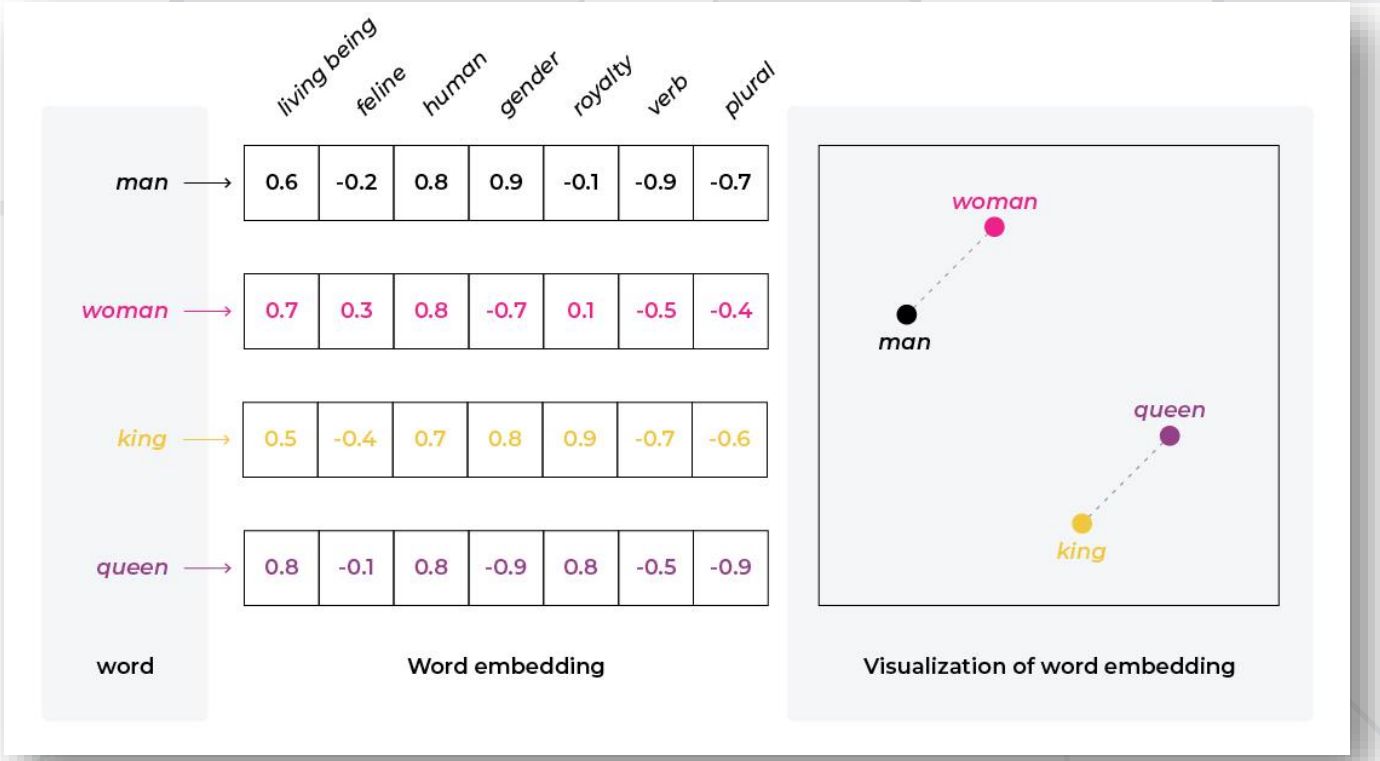
What Is a Vector

- A **list** of **numbers** (e.g. [0.12, -0.98, ...])
- Represents something in space
- The closer the vectors → the more similar
- Used for **comparing meaning**
- Powers search and matching

```
# Example vectors of different lengths
vector_2d = [3, 4]
vector_3d = [1, 2, 3]
vector_5d = [0.1, 0.5, 0.3, 0.7, 0.9]
```

What Are Embeddings

- A way to represent **meaning** with **numbers**
- Text turns into a list of numbers (**vector**)
- **Similar** ideas → similar numbers
- Used in **search**, **chatbots**, **recommendations**
- Makes AI "**understand**" content



From Text to Vector

- **Input**: word, sentence, or document
- Output: numeric vector
- Captures **meaning**, not just keywords
- **Example**: "return product" ≈ "send item back"
- **Vectors are compared, not the raw text**

```
Input text: Embedding turns text into a vector
Vector shape: (384,)
Vector (first 5 values): [ 0.0375  -0.0923   0.0281   0.0714  -0.0447]
```

How Are Embeddings Generated

- Sent to a pre-trained **AI model**
- The model returns a vector
- One call = one embedding
- No training required

```python
import openai

openai.api_key = "YOUR_API_KEY"

response = openai.embeddings.create(
    input="Embedding turns text into a vector.",
    model="text-embedding-3-small"
)

vector = response.data[0].embedding
```

Comparing Meaning

- Two **similar** texts → close vectors
- **Different** texts → distant vectors
- The search checks "how close" they are
- Helps rank results by relevance
- Enables smarter search and answers

How Vectors Are Compared

Software University

- Distance = difference in meaning
- Smaller distance = more similar content
- Most common method : **cosine similarity**
- Result is a number from -1 to 1
- Closer to 1 = more similar

```
def cosine_similarity(vec1, vec2):
    v1 = np.array(vec1)
    v2 = np.array(vec2)
    return np.dot(v1, v2) / (np.linalg.norm(v1) * np.linalg.norm(v2))

text1 = "How do I cook pasta?"
text2 = "What's the best way to boil spaghetti?"

embedding1 = get_embedding(text1)
embedding2 = get_embedding(text2)

similarity = cosine_similarity(embedding1, embedding2)

print(f"Similarity: {similarity:.4f}")
```

Where Are Embeddings Used

- **AI chatbots** (to find answers)
- Search engines (semantic search)
- Grouping similar documents
- Matching user **queries** to **content**

Can We Embed Long Text?

Software University

- You can embed text of different **lengths**
- Output vector size is **always the same** (e.g. 1536)
- There is a token **limit**
- Longer texts must be split into **chunks**

```
texts = [
    "Hello",  # short
    "How are you doing today?",  # medium
    "In machine learning, embeddings are used to represent text as dense vectors."  # long
]

for text in texts:
    response = openai.embeddings.create(
        input=text,
        model="text-embedding-3-small"
    )
    vector = response.data[0].embedding
```



Vector Databases

How AI Remembers and Finds Information

What Is a Vector Database

- Stores and indexes **embedding vectors**
- Supports fast similarity search
- Finds the closest **vector** to a **query**
- Works with **high-dimensional data**
- Used in AI search, chatbots, recommendations

Why Not Use SQL or NoSQL

Software University

- SQL / NoSQL **can't search** by meaning
- They match exact text or keywords
- Vectors need **mathematical similarity** (not equality)
- Vector DBs use distance metrics instead
- Ideal for fuzzy, semantic queries

Chroma

- **Python-native** vector database
- Perfect for demos and **small** apps
- No external server or API needed
- Stores vectors and original texts
- **Easy** to use in Google Colab

```
collection.upsert(
    documents=[
        "This is a document about pineapple",
        "This is a document about oranges"
    ],
    ids=["id1", "id2"]
)

results = collection.query(
    query_texts=["This is a query document about florida"],
    n_results=2 # how many results to return
)
```

Setup Chroma

Software University

- Setup and installation
- Collections

```
pip install chromadb
```

```
import chromadb
chroma_client = chromadb.Client()
```

```
collection = chroma_client.get_or_create_collection(name="my_collection")

# switch `add` to `upsert` to avoid adding the same documents every time
collection.upsert(
    documents=[
        "This is a document about pineapple",
        "This is a document about oranges"
    ],
    ids=["id1", "id2"]
)

results = collection.query(
    query_texts=["This is a query document about florida"], # Chroma will embed this for you
    n_results=2 # how many results to return
)
```

Run Chroma

- In-memory
- Locally
- On a server
- In the cloud

```
client = chromadb.EphemeralClient()
```

```
client = chromadb.PersistentClient(path="/path/to/save/to")
```

```
chroma_client = chromadb.HttpClient(host='localhost', port=8000)
```

```
client = CloudClient(
    tenant='Tenant ID',
    database='Database name',
    api_key='Chroma Cloud API key'
)
```

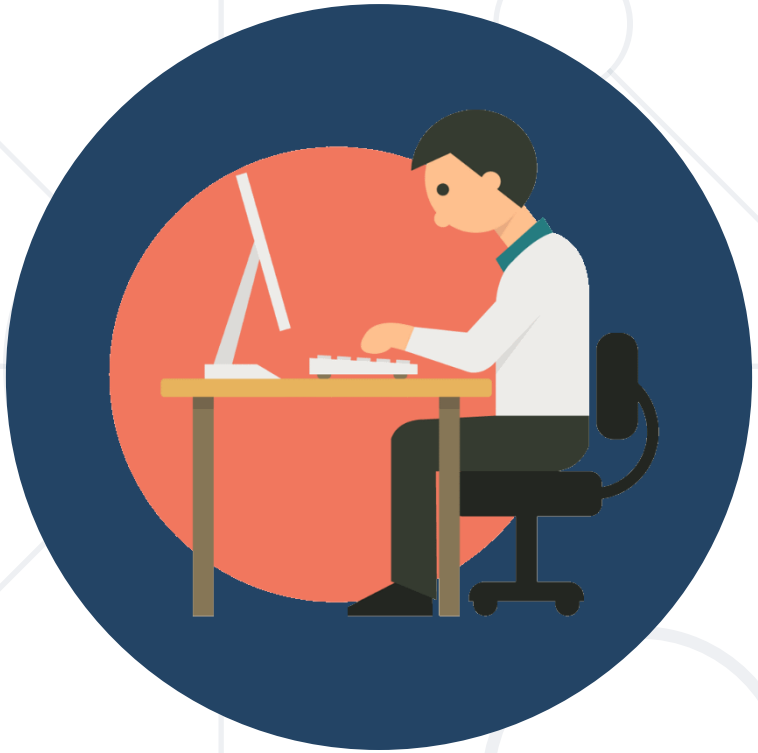
Collections in Chroma

- Unique names
- Different embedding functions
- Metadata
- Can add, update and delete data

```
collection = client.create_collection(
    name="my_collection",
    embedding_function=emb_fn,
    metadata={
        "description": "my first Chroma collection",
        "created": str(datetime.now())
    }
)
```

Popular Vector Databases

- Chroma – lightweight, Python-native
- FAISS – fast, local, open source
- **Pinecone** – scalable, fully managed
- Weaviate – hybrid search with metadata
- Milvus – high performance, GPU support



Practice

Exercise – Play with Chroma



RAG

Giving LLMs Access to Real Knowledge

What Is RAG

- Stands for **Retrieval-Augmented Generation**
- Combines search with text generation
- LLM gets real info from **external sources**
- No need to train the model
- Used in **AI chatbots**, assistants, Q&A systems

Why RAG Exists

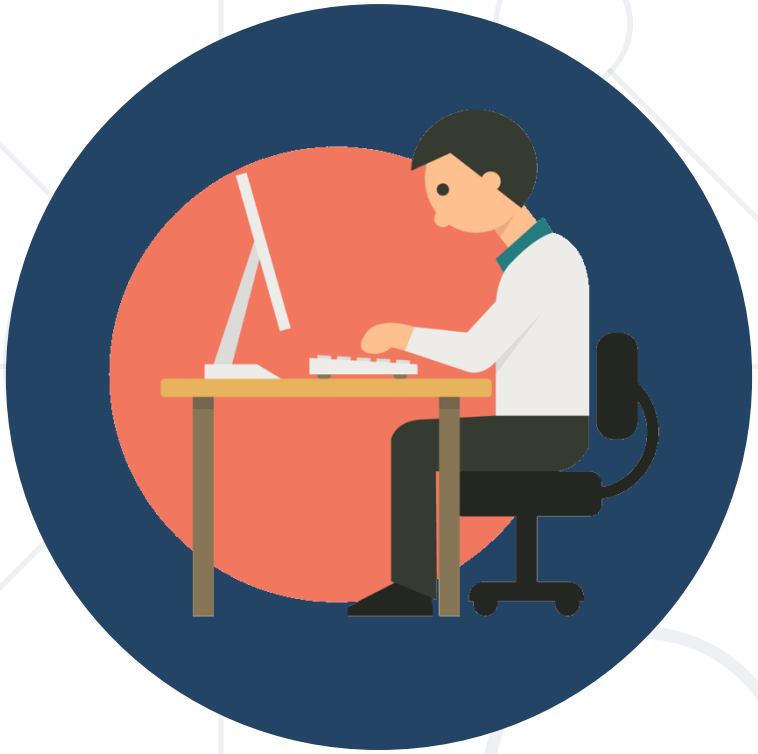
- LLMs **don't know** recent or specific data
- Sometimes we need **dynamic** knowledge
- RAG gives LLMs access to **external facts**
- Makes answers more accurate and useful

Example – Return Policy Chatbot

- User: "How can I return a product?"
- We search company documents
- Find paragraph with return policy
- Send it to GPT with the question
- Bot answers with correct company rules

How It All Connects

- Embeddings represent **meaning**
- Vector DB **stores** and **finds** related info
- **RAG links** that info to a smart LLM prompt
- The model responds like it "knows" the source
- This is the **foundation** of custom AI chatbots



Practice

Exercise – End-to-end Implementation



BONUS: OpenAI RAG

Built-in Retrieval

OpenAI RAG

- Upload documents **directly** to OpenAI
- LLM searches and reads from them
- **No need** to create embeddings manually
- **No vector DB** or chunking required
- Powered by the **Assistants API**

How It Works

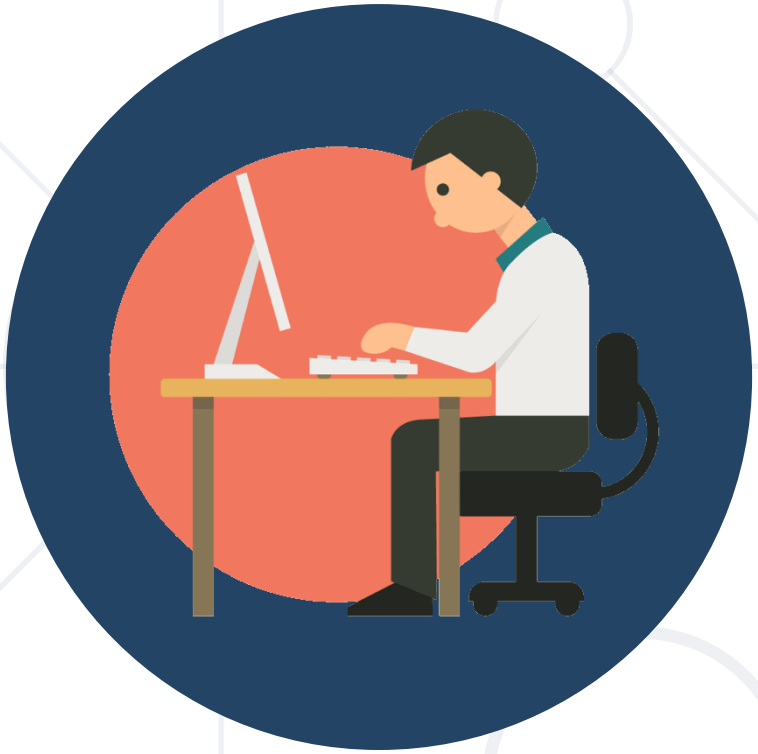
- Upload file (PDF, TXT, etc.)
- Ask a question – LLM does the rest
- Retrieval happens behind the scenes
- Works for small sets of documents

Benefits of Built-in Retrieval

- **Fast** and **simple** setup
- **Less code** and **no infrastructure**
- No need to manage chunks or vector stores
- Great for **quick prototypes** or **demos**
- Ideal for **non-technical teams**

Limitations

- **No transparency** into what was retrieved
- **Limited** customization
- Cloud-only – files go to OpenAI servers



Practice

Exercise – End-to-end Implementation

Summary

- Embeddings turn text into vectors that capture meaning
- Vector databases enable fast semantic search
- RAG combines retrieval and generation for smarter answers
- OpenAI Retrieval offers a simpler, fast alternative with less control



Trainings @ Software University (SoftUni)

- Software University – High-Quality Education, Profession and Job for Software Developers
  - softuni.bg
- Software University Foundation
  - softuni.foundation
- Software University @ Facebook
  - facebook.com/SoftwareUniversity



License

- This course (slides, examples, demos, exercises, homework, documents, videos and other assets) is **copyrighted content**
- Unauthorized copy, reproduction or use is illegal
- © SoftUni – https://about.softuni.bg
- © Software University – https://softuni.bg

